

Cours de Sciences de Données avec Python

Fatoumata Diallo

Table des matières

1	Introduction à Python pour la Data Science	4
2	Variables et opérations de base	4
2.1	Variables et types	4
2.2	Opérations arithmétiques	4
2.3	Opérateurs logiques et booléens	5
2.4	Exemple	5
3	Conditions et boucles	5
3.1	Structures conditionnelles	5
3.2	Boucle for	6
3.3	Boucle while	6
3.4	Exemple	6
3.5	Exercice	6
4	Fonctions en Python	6
4.1	Exemple : fonction sans paramètre	7
4.2	Fonction avec paramètres et retour de valeur	7
4.3	Paramètres par défaut	7
4.4	Arguments	7
4.5	Fonctions anonymes (lambda)	7
4.6	Exemple	8
5	Structures de données en Python	8
5.1	Les listes	8
5.2	Les tuples	9
5.3	Les dictionnaires	9
5.4	Les ensembles (sets)	10
5.5	Compréhensions de listes et de dictionnaires	10
5.6	Exemple	10
6	Opérations sur les structures de données	10
6.1	Opérations sur les listes	10
6.2	Opérations sur les tuples	11
6.3	Opérations sur les dictionnaires	11
6.4	Opérations sur les ensembles	12
6.5	Conversions entre structures	12
6.6	Exemple	12

7	Manipulation avancée des chaînes de caractères	13
7.1	Définition	13
7.2	Indexation et slicing	13
7.3	Concaténation et répétition	13
7.4	Méthodes utiles des chaînes	13
7.5	Itération sur une chaîne	14
7.6	Test d'appartenance	14
7.7	Formatage des chaînes	14
7.8	Exemple	14
8	Entrées et sorties (I/O)	14
8.1	Lecture et écriture dans la console	14
8.2	Lecture et écriture dans les fichiers	15
8.3	Lecture ligne par ligne	15
8.4	Écriture en mode ajout	15
8.5	Vérification de l'existence d'un fichier	16
8.6	Exemple	16
9	Modules et packages en Python	16
9.1	Définition	16
9.2	Importer un module	16
9.3	Modules utiles de la bibliothèque standard	17
9.4	Création d'un module personnalisé	17
9.5	Installation de packages externes	17
9.6	Exemple	18
10	NumPy : Calcul scientifique	18
10.1	Introduction à NumPy	18
10.2	Création de tableaux	18
10.3	Génération de séquences numériques	19
10.4	Dimensions et formes	19
10.5	Indexation et slicing	19
10.6	Broadcasting	19
10.7	Opérations mathématiques	19
10.8	Statistiques de base	20
10.9	Algèbre linéaire	20
10.10	Gestion des valeurs manquantes (NaN)	20
10.11	Exemple	20
11	Visualisation des données avec Matplotlib	21
11.1	Tracé de courbes avec <code>plot</code>	21
11.2	Nuage de points (<code>scatter</code>)	21
11.3	Plusieurs graphiques dans une figure	22
11.4	Interface orientée objet	23
11.5	Histogrammes	24
11.6	Histogramme 2D	25
11.7	Courbes de niveau (<code>contour</code>)	26
11.8	Graphiques 3D	26
11.9	Matrice de corrélation avec <code>imshow</code>	27

12 Visualisation avancée avec Seaborn	28
12.1 Exemple avec iris	28
12.2 Visualisation avec pairplot	29
12.3 Exemples avec le dataset Titanic	30
12.4 Boxplots	30
12.5 Distribution des variables	31
12.6 Heatmap des corrélations	31
12.7 Fonctions principales	32
13 Machine Learning : Apprentissage supervisé	32
13.1 Introduction	32
13.2 Apprentissage supervisé	33
14 La bibliothèque Scikit-Learn	33
14.1 Syntaxe générale	33
15 Évaluation et amélioration des modèles	34
15.1 Trainset et Testset	34
15.2 Validation Set et Cross Validation	35
15.3 Validation Curve	35
15.4 GridSearchCV	35
15.5 Learning Curve	36

1 Introduction à Python pour la Data Science

Python est l'un des langages les plus utilisés en **science des données** pour sa simplicité et l'écosystème de bibliothèques : NumPy (calcul numérique), Pandas (tableaux), Matplotlib/Seaborn (visualisation) et scikit-learn (apprentissage automatique).

Un premier programme très simple consiste à afficher un message à l'écran :

```
1 print("Bienvenue dans le cours de Python pour la Data Science !")
```

2 Variables et opérations de base

2.1 Variables et types

Une **variable** est un espace mémoire qui contient une valeur. En Python, il n'est pas nécessaire de déclarer le type d'une variable : il est détecté automatiquement (typage dynamique).

Les types fondamentaux sont :

- **int** : nombres entiers,
- **float** : nombres réels,
- **str** : chaînes de caractères,
- **bool** : valeurs logiques True/False.

```
1 x = 10          # entier (int)
2 y = 3.14        # flottant (float)
3 nom = "Fatou"   # chaîne (str)
4 valide = True   # booléen (bool)
5
6 print(type(x), type(y), type(nom), type(valide))
```

2.2 Opérations arithmétiques

Les opérations de base sont :

- **+** : addition,
- **-** : soustraction,
- ***** : multiplication,
- **/** : division réelle,
- **//** : division entière,
- **%** : modulo (reste),
- ****** : puissance.

```
1 a = 7
2 b = 2
3
4 print("Addition :", a + b)
5 print("Soustraction :", a - b)
6 print("Multiplication :", a * b)
7 print("Division :", a / b)          # 3.5
8 print("Division entière :", a // b) # 3
9 print("Modulo :", a % b)           # 1
10 print("Puissance :", a ** b)       # 49
```

2.3 Opérateurs logiques et booléens

Les opérateurs de comparaison permettent de tester des conditions :

- `==` : égalité,
- `!=` : différence,
- `>` : strictement supérieur,
- `<` : strictement inférieur,
- `>=`, `<=` : supérieur ou égal / inférieur ou égal.

Ils se combinent avec des opérateurs logiques :

- `and` : ET logique,
- `or` : OU logique,
- `not` : négation.

```
1 x = 5
2 y = 10
3
4 print(x > y)          # False
5 print(x < y)          # True
6 print(x == 5)         # True
7 print(x != y)         # True
8
9 print(x > 0 and y > 0) # True
10 print(x > 0 or y < 0) # True
11 print(not (x > y))    # True
```

2.4 Exemple

Écrire un programme qui calcule l'aire d'un rectangle.

```
1 longueur = 5
2 largeur = 3
3 aire = longueur * largeur
4 print("Aire =", aire)
```

3 Conditions et boucles

3.1 Structures conditionnelles

Les structures conditionnelles permettent d'exécuter du code en fonction d'un test logique. La syntaxe est :

- `if` : si la condition est vraie,
- `elif` : sinon si une autre condition est vraie,
- `else` : sinon (aucune condition remplie).

```
1 x = 2
2
3 if x > 0:
4     print("x est positif")
5 elif x == 0:
6     print("x est nul")
```

```
7 else:
8     print("x est négatif")
```

3.2 Boucle for

La boucle `for` permet de répéter un bloc d'instructions pour chaque élément d'une séquence ou d'un intervalle numérique.

```
1 for i in range(5):
2     print("Itération", i)
```

3.3 Boucle while

La boucle `while` exécute un bloc tant qu'une condition reste vraie.

```
1 x = 0
2 while x < 5:
3     print("x =", x)
4     x += 1
```

3.4 Exemple

Programme qui indique si un nombre est pair ou impair.

```
1 n = int(input("Entrez un nombre : "))
2 if n % 2 == 0:
3     print(n, "est pair")
4 else:
5     print(n, "est impair")
```

3.5 Exercice

Calculer la somme des 10 premiers entiers positifs.

```
1 somme = 0
2 for i in range(1, 11):
3     somme += i
4 print("Somme des 10 premiers entiers =", somme)
```

4 Fonctions en Python

Une **fonction** est un bloc de code réutilisable qui effectue une tâche précise. Elle permet :

- de réduire la duplication du code,
- de structurer et organiser les programmes,
- de rendre le code plus lisible et maintenable.

En Python, une fonction se définit avec le mot-clé `def`. La syntaxe générale est :

```

1 def nom_de_fonction(parametres):
2     # bloc d'instructions
3     return resultat

```

4.1 Exemple : fonction sans paramètre

```

1 def saluer():
2     print("Bonjour, bienvenue en Python !")
3
4 saluer()

```

4.2 Fonction avec paramètres et retour de valeur

```

1 def carre(x):
2     return x**2
3
4 print(carre(4))    # 16
5 print(carre(10))   # 100

```

4.3 Paramètres par défaut

```

1 def presentation(nom, age=18):
2     print("Je m'appelle", nom, "et j'ai", age, "ans.")
3
4 presentation("Fatou", 25)
5 presentation("Julie")    # âge par défaut = 18

```

4.4 Arguments

Les arguments peuvent être passés :

- par position,
- par nom.

```

1 def addition(a, b):
2     return a + b
3
4 print(addition(5, 3))    # positionnels
5 print(addition(a=10, b=7)) # nommés
6 print(addition(b=4, a=6)) # ordre inversé possible

```

4.5 Fonctions anonymes (lambda)

Les fonctions `lambda` sont des fonctions courtes, écrites en une seule ligne. Elles sont souvent utilisées comme argument dans d'autres fonctions.

```

1 f = lambda x: x**2
2 print(f(5))    # 25
3
4 produit = lambda x, y: x * y
5 print(produit(4, 3))    # 12

```

4.6 Exemple

Écrire une fonction qui calcule la moyenne d'une liste de nombres.

```

1 def moyenne(liste):
2     return sum(liste) / len(liste)
3
4 notes = [12, 15, 9, 18, 14]
5 print("Moyenne :", moyenne(notes))

```

1. Écrire une fonction factorielle(n) qui calcule $n! = 1 \times 2 \times \dots \times n$. 2. Écrire une fonction est_pair(n) qui retourne True si n est pair, False sinon.

```

1 def factorielle(n):
2     res = 1
3     for i in range(1, n+1):
4         res *= i
5     return res
6
7 def est_pair(n):
8     return n % 2 == 0
9
10 print(factorielle(5))    # 120
11 print(est_pair(7))      # False
12 print(est_pair(10))     # True

```

5 Structures de données en Python

5.1 Les listes

Une **liste** est une structure de données ordonnée et modifiable (mutable). Elle peut contenir des éléments de différents types.

```

1 # Création de listes
2 nombres = [1, 2, 3, 4, 5]
3 melange = [10, "Python", 3.14, True]
4
5 # Indexation
6 print(nombres[0])    # premier élément
7 print(nombres[-1])   # dernier élément
8
9 # Slicing (tranches)
10 print(nombres[1:4])  # éléments d'indices 1 à 3
11 print(nombres[:3])   # du début jusqu'à l'indice 2

```



```
12 print(nombres[::2])    # un élément sur deux
```

Opérations principales sur les listes :

```
1 villes = ["Paris", "Berlin", "Londres"]
2
3 villes.append("Rome")      # ajout en fin
4 villes.insert(1, "Madrid") # insertion à un indice
5 villes.remove("Paris")    # suppression d'un élément
6 dernier = villes.pop()    # enlève et retourne le dernier élément
7
8 print(len(villes))        # taille de la liste
9 print("Berlin" in villes) # test d'appartenance
10
11 villes.sort()            # tri alphabétique
12 villes.reverse()         # inversion de l'ordre
```

5.2 Les tuples

Un **tuple** est similaire à une liste mais **immutable** : ses éléments ne peuvent pas être modifiés après création.

```
1 coordonnees = (10, 20)
2 print(coordonnees[0])    # accès par indice
3
4 # tuple d'un seul élément
5 t = (5,)
```

Les tuples sont utiles pour représenter des données fixes (par exemple, des coordonnées géographiques).

5.3 Les dictionnaires

Un **dictionnaire** est une collection non ordonnée d'associations **clé:valeur**. Les clés doivent être uniques et immuables (str, int, tuple).

```
1 etudiant = {
2     "nom": "Fatou",
3     "age": 25,
4     "note": 16
5 }
6
7 print(etudiant["nom"])
8 etudiant["note"] = 18
9 etudiant["ville"] = "Paris"
10
11 print(etudiant.keys())
12 print(etudiant.values())
13 print(etudiant.items())
```

On peut parcourir un dictionnaire avec une boucle :

```

1 for cle, valeur in etudiant.items():
2     print(cle, ":", valeur)

```

5.4 Les ensembles (sets)

Un **set** est une collection non ordonnée et sans doublons. Il est très utile pour effectuer des opérations ensemblistes.

```

1 A = {1, 2, 3, 4}
2 B = {3, 4, 5, 6}
3
4 print(A | B)    # union
5 print(A & B)    # intersection
6 print(A - B)    # différence
7 print(A ^ B)    # différence symétrique

```

5.5 Compréhensions de listes et de dictionnaires

Les **compréhensions** permettent de créer des listes, ensembles ou dictionnaires en une seule ligne.

```

1 carres = [x**2 for x in range(6)]
2 print(carres) # [0, 1, 4, 9, 16, 25]
3
4 pairs = [x for x in range(10) if x % 2 == 0]
5 print(pairs)  # [0, 2, 4, 6, 8]
6
7 dico = {x: x**2 for x in range(4)}
8 print(dico)   # {0: 0, 1: 1, 2: 4, 3: 9}

```

5.6 Exemple

1. Créer une liste de nombres de 1 à 20. 2. Extraire les carrés des nombres pairs dans une nouvelle liste. 3. Enregistrer le résultat dans un dictionnaire où la clé est le nombre et la valeur son carré.

```

1 nombres = list(range(1, 21))
2 carres_pairs = {x: x**2 for x in nombres if x % 2 == 0}
3 print(carres_pairs)

```

6 Opérations sur les structures de données

6.1 Opérations sur les listes

Les listes offrent un grand nombre de méthodes intégrées permettant de manipuler facilement des données.

— `append(x)` : ajoute un élément en fin de liste.

- `insert(i, x)` : insère un élément à une position donnée.
- `extend(L)` : étend une liste avec une autre liste.
- `remove(x)` : supprime la première occurrence d'un élément.
- `pop(i)` : supprime et retourne l'élément à la position `i` (ou le dernier si vide).
- `count(x)` : compte le nombre d'occurrences d'un élément.
- `index(x)` : retourne l'indice de la première occurrence.
- `sort()` : trie la liste.
- `reverse()` : inverse l'ordre des éléments.

```

1 villes = ["Paris", "Berlin", "Londres", "Madrid"]
2
3 villes.append("Rome")
4 villes.insert(1, "Bruxelles")
5 villes.remove("Paris")
6 dernier = villes.pop()
7
8 print("Liste finale :", villes)
9 print("Dernier élément retiré :", dernier)

```

6.2 Opérations sur les tuples

Les tuples étant immuables, les opérations sont limitées :

- accès par indice,
- découpage (slicing),
- concaténation avec `+`,
- répétition avec `*`.

```

1 t1 = (1, 2, 3)
2 t2 = (4, 5)
3
4 print(t1 + t2)    # concaténation
5 print(t1 * 2)     # répétition
6 print(t1[1:])     # slicing

```

6.3 Opérations sur les dictionnaires

Les dictionnaires possèdent des méthodes puissantes pour gérer les données clé/valeur.

- `get(cle, default)` : retourne la valeur associée à une clé (ou une valeur par défaut).
- `update(dict)` : met à jour avec un autre dictionnaire.
- `pop(cle)` : supprime et retourne une valeur.
- `popitem()` : supprime et retourne la dernière paire clé/valeur.
- `clear()` : vide le dictionnaire.

```

1 etudiant = {"nom": "Fatou", "age": 25, "note": 16}
2
3 print(etudiant.get("ville", "Inconnue"))
4 etudiant.update({"ville": "Paris", "age": 26})
5 print(etudiant)
6

```

```

7 note = etudiant.pop("note")
8 print("Note supprimée :", note)
9
10 print("Clés :", etudiant.keys())
11 print("Valeurs :", etudiant.values())

```

6.4 Opérations sur les ensembles

Les ensembles permettent les opérations classiques de la théorie des ensembles.

```

1 A = {1, 2, 3, 4}
2 B = {3, 4, 5, 6}
3
4 print("Union :", A | B)
5 print("Intersection :", A & B)
6 print("Différence :", A - B)
7 print("Différence symétrique :", A ^ B)
8
9 A.add(7)           # ajout
10 B.remove(6)       # suppression
11 print(A, B)

```

6.5 Conversions entre structures

Python permet de convertir facilement entre différents types de structures.

```

1 liste = [1, 2, 2, 3]
2 ensemble = set(liste)           # suppression des doublons
3 print(ensemble)                 # {1, 2, 3}
4
5 dico = dict([(1, "un"), (2, "deux")])
6 print(dico)                    # {1: 'un', 2: 'deux'}
7
8 liste2 = list(dico.keys())
9 print(liste2)                  # [1, 2]

```

6.6 Exemple

1. Créer une liste de prénoms avec des doublons. 2. Supprimer les doublons grâce à un ensemble. 3. Construire un dictionnaire associant chaque prénom à sa longueur.

```

1 prenom = ["Julie", "Marc", "Julie", "Sophie", "Marc"]
2 unique = set(prenom)
3
4 dico = {nom: len(nom) for nom in unique}
5 print(dico)

```

7 Manipulation avancée des chaînes de caractères

7.1 Définition

Une **chaîne de caractères** (type `str`) est une séquence immuable de symboles (lettres, chiffres, espaces, ponctuation). En Python, les chaînes sont entourées de guillemets simples ou doubles.

```
1 texte1 = "Bonjour"
2 texte2 = 'Python'
```

7.2 Indexation et slicing

Comme les listes, les chaînes sont indexées à partir de 0 et supportent le découpage (slicing).

```
1 mot = "Python"
2
3 print(mot[0])      # P
4 print(mot[-1])     # n
5 print(mot[1:4])    # yth
6 print(mot[:3])     # Pyt
7 print(mot[::-1])   # nohtyP (inversé)
```

7.3 Concaténation et répétition

```
1 prenom = "Fatou"
2 nom = "Diallo"
3
4 nom_complet = prenom + " " + nom
5 print(nom_complet)
6
7 rire = "ha" * 3
8 print(rire)      # hahaha
```

7.4 Méthodes utiles des chaînes

- `lower()` : met en minuscules,
- `upper()` : met en majuscules,
- `capitalize()` : met la première lettre en majuscule,
- `strip()` : supprime les espaces au début et à la fin,
- `replace(a, b)` : remplace `a` par `b`,
- `split(sep)` : découpe selon un séparateur,
- `join(liste)` : concatène les éléments d'une liste avec un séparateur.

```
1 phrase = " Python est Génial "
2 print(phrase.lower())
3 print(phrase.upper())
4 print(phrase.strip())
5 print(phrase.replace("Génial", "puissant"))
```

```

6
7 mots = phrase.split()
8 print(mots)
9 print("-".join(mots))

```

7.5 Itération sur une chaîne

```

1 for lettre in "Python":
2     print(lettre)

```

7.6 Test d'appartenance

```

1 texte = "Machine Learning"
2 print("Learn" in texte)      # True
3 print("Deep" not in texte)   # True

```

7.7 Formatage des chaînes

Python propose plusieurs manières d'insérer des valeurs dans une chaîne.

```

1 nom = "Fatou"
2 age = 25
3
4 # Méthode format
5 print("Je m'appelle {} et j'ai {} ans".format(nom, age))
6
7 # f-strings (recommandé depuis Python 3.6)
8 print(f"Je m'appelle {nom} et j'ai {age} ans")

```

7.8 Exemple

1. Demander à l'utilisateur son prénom. 2. Afficher un message personnalisé en majuscules. 3. Indiquer le nombre de lettres dans le prénom.

```

1 prenom = input("Entrez votre prénom : ")
2
3 print("Bonjour", prenom.upper())
4 print("Votre prénom contient", len(prenom), "lettres")

```

8 Entrées et sorties (I/O)

8.1 Lecture et écriture dans la console

En Python, la fonction `print()` permet d'afficher un message, tandis que la fonction `input()` permet de saisir des données depuis le clavier.

```

1 # Affichage simple
2 print("Bienvenue dans le cours de Python !")
3
4 # Lecture d'une entrée utilisateur
5 nom = input("Entrez votre nom : ")
6 print("Bonjour", nom)

```

Par défaut, la fonction `input()` retourne une chaîne de caractères. Il faut donc convertir en entier ou flottant si nécessaire.

```

1 age = int(input("Entrez votre âge : "))
2 taille = float(input("Entrez votre taille en mètres : "))
3
4 print(f"Vous avez {age} ans et mesurez {taille} m")

```

8.2 Lecture et écriture dans les fichiers

La fonction `open()` permet d'ouvrir un fichier. On doit préciser le nom du fichier et le mode :

- "r" : lecture,
- "w" : écriture (écrase si le fichier existe),
- "a" : ajout en fin de fichier,
- "b" : mode binaire (optionnel).

```

1 # Écriture dans un fichier
2 with open("exemple.txt", "w", encoding="utf-8") as f:
3     f.write("Bonjour Python\n")
4     f.write("Une deuxième ligne\n")
5
6 # Lecture du fichier
7 with open("exemple.txt", "r", encoding="utf-8") as f:
8     contenu = f.read()
9     print(contenu)

```

8.3 Lecture ligne par ligne

```

1 with open("exemple.txt", "r", encoding="utf-8") as f:
2     for ligne in f:
3         print("Ligne :", ligne.strip())

```

8.4 Écriture en mode ajout

```

1 with open("exemple.txt", "a", encoding="utf-8") as f:
2     f.write("Ajout d'une nouvelle ligne\n")

```

8.5 Vérification de l'existence d'un fichier

Le module `os` permet de gérer les fichiers et dossiers.

```
1 import os
2
3 if os.path.exists("exemple.txt"):
4     print("Le fichier existe")
5 else:
6     print("Fichier introuvable")
```

8.6 Exemple

1. Demander à l'utilisateur son prénom et son âge. 2. Écrire ces informations dans un fichier texte. 3. Lire le fichier et afficher le contenu.

```
1 nom = input("Entrez votre prénom : ")
2 age = input("Entrez votre âge : ")
3
4 with open("utilisateur.txt", "w", encoding="utf-8") as f:
5     f.write(f"Nom : {nom}\n")
6     f.write(f"Âge : {age}\n")
7
8 with open("utilisateur.txt", "r", encoding="utf-8") as f:
9     print(f.read())
```

9 Modules et packages en Python

9.1 Définition

Un **module** est un fichier Python contenant des fonctions, classes ou variables réutilisables. Un **package** est un ensemble organisé de modules regroupés dans un dossier, souvent avec un fichier `__init__.py`.

Python propose une large bibliothèque standard, et il est possible d'installer des bibliothèques externes avec `pip`.

9.2 Importer un module

```
1 import math
2
3 print(math.sqrt(16))      # racine carrée
4 print(math.pi)           # constante pi
```

On peut aussi importer uniquement certaines fonctions :

```
1 from math import sin, cos
2
3 print(sin(0))
4 print(cos(0))
```


Ou encore donner un alias pour simplifier l'écriture :

```
1 import numpy as np
2
3 A = np.array([1, 2, 3])
4 print(A * 2)
```

9.3 Modules utiles de la bibliothèque standard

- `math` : fonctions mathématiques,
- `random` : génération aléatoire,
- `statistics` : calculs statistiques,
- `os` : gestion des fichiers et dossiers,
- `glob` : recherche de fichiers avec motifs,
- `datetime` : gestion des dates et heures.

```
1 import random, statistics
2
3 liste = [1, 2, 3, 4, 5]
4
5 print(random.choice(liste))      # élément aléatoire
6 print(random.sample(liste, 3))   # échantillon sans remise
7 print(statistics.mean(liste))    # moyenne
```

9.4 Création d'un module personnalisé

Il est possible d'écrire ses propres modules. Par exemple, créer un fichier `utilitaires.py` :

```
1 # fichier utilitaires.py
2 def salutation():
3     return "Bonjour depuis le module utilitaires"
```

Puis l'utiliser dans un autre script :

```
1 import utilitaires
2
3 print(utilitaires.salutation())
```

9.5 Installation de packages externes

Les bibliothèques externes s'installent avec `pip`. Exemple pour installer NumPy :

```
1 pip install numpy
```

Puis utilisation :

```
1 import numpy as np
2
3 A = np.arange(0, 10, 2)
4 print(A)
```

9.6 Exemple

1. Importer le module `math` et calculer la valeur de $\sin(\pi/4)$. 2. Générer un nombre entier aléatoire entre 1 et 100 avec `random`. 3. Créer un module `calculs.py` contenant une fonction `cube(n)` et l'utiliser dans un script.

```
1 import math, random
2
3 print("sin(pi/4) =", math.sin(math.pi/4))
4 print("Nombre aléatoire :", random.randint(1, 100))
5
6 # fichier calculs.py
7 def cube(n):
8     return n**3
9
10 # fichier principal
11 import calculs
12 print(calculs.cube(3))
```

10 NumPy : Calcul scientifique

10.1 Introduction à NumPy

NumPy (Numerical Python) est une bibliothèque fondamentale pour le calcul scientifique. Elle fournit :

- le type `ndarray`, tableau multidimensionnel efficace,
- des fonctions rapides pour le calcul vectorisé,
- des outils d'algèbre linéaire, statistiques et manipulation de données.
-

10.2 Création de tableaux

```
1 import numpy as np
2
3 # À partir d'une liste Python
4 A = np.array([1, 2, 3, 4])
5 print(A)
6
7 # Tableaux multidimensionnels
8 B = np.array([[1, 2, 3], [4, 5, 6]])
9 print(B)
10
11 # Tableaux spéciaux
12 zeros = np.zeros((3, 3))
13 ones = np.ones((2, 4))
14 full = np.full((2, 2), 7)
15 identite = np.eye(3)
16
17 print(zeros)
18 print(identite)
```

10.3 Génération de séquences numériques

```
1 # Arange : intervalle
2 x = np.arange(0, 10, 2)    # [0 2 4 6 8]
3
4 # Linspace : valeurs régulièrement espacées
5 y = np.linspace(0, 1, 5)  # [0.  0.25 0.5  0.75 1. ]
```

10.4 Dimensions et formes

```
1 A = np.array([[1, 2, 3], [4, 5, 6]])
2
3 print(A.shape)    # dimensions (2, 3)
4 print(A.ndim)     # nombre d'axes (2)
5 print(A.size)     # nombre total d'éléments (6)
```

10.5 Indexation et slicing

```
1 A = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
2
3 print(A[0, 1])    # élément ligne 0, colonne 1
4 print(A[:, 1])    # toutes les lignes, colonne 1
5 print(A[1, :])    # ligne 1 complète
6 print(A[0:2, 0:2]) # sous-matrice
```

10.6 Broadcasting

Le **broadcasting** permet d'effectuer des opérations entre tableaux de formes différentes, tant que les dimensions sont compatibles.

```
1 A = np.array([[1, 2, 3], [4, 5, 6]])
2 B = np.array([10, 20, 30])
3
4 print(A + B)
5 # [[11 22 33]
6 #  [14 25 36]]
```

10.7 Opérations mathématiques

```
1 A = np.array([1, 2, 3, 4])
2
3 print(np.sqrt(A))
4 print(np.exp(A))
5 print(np.log(A))
6 print(np.sin(A))
```

10.8 Statistiques de base

```
1 A = np.random.randint(0, 10, (3, 4))
2
3 print("Moyenne :", A.mean())
4 print("Variance :", np.var(A))
5 print("Écart-type :", np.std(A))
6 print("Somme par colonne :", A.sum(axis=0))
7 print("Somme par ligne :", A.sum(axis=1))
```

10.9 Algèbre linéaire

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 print("Produit matriciel :")
5 print(A.dot(B))
6
7 # Déterminant et inverse
8 print("Determinant :", np.linalg.det(A))
9 print("Inverse :\n", np.linalg.inv(A))
10
11 # Valeurs propres et vecteurs propres
12 valeurs, vecteurs = np.linalg.eig(A)
13 print("Valeurs propres :", valeurs)
14 print("Vecteurs propres :\n", vecteurs)
```

10.10 Gestion des valeurs manquantes (NaN)

```
1 A = np.array([1, 2, np.nan, 4])
2
3 print("Moyenne classique :", np.mean(A))           # NaN
4 print("Moyenne corrigée :", np.nanmean(A))         # ignore les NaN
5 print("Nombre de NaN :", np.isnan(A).sum())
```

10.11 Exemple

1. Créer une matrice aléatoire 5×5 d'entiers entre 0 et 20. 2. Remplacer tous les éléments < 10 par 0. 3. Calculer la moyenne de chaque ligne.

```
1 np.random.seed(0)
2 M = np.random.randint(0, 20, (5, 5))
3
4 M[M < 10] = 0
5 print(M)
6
7 print("Moyenne par ligne :", M.mean(axis=1))
```

11 Visualisation des données avec Matplotlib

La visualisation des données transforme des chiffres en graphiques pour révéler *tendances*, *relations* et *structures*. Avec **Matplotlib**, on produit des tracés 2D/3D, personnalise styles et annote les figures.

11.1 Tracé de courbes avec plot

`plot()` relie des points (x_i, y_i) pour visualiser des fonctions ou séries temporelles. Paramètres clés : `c` (couleur), `lw` (épaisseur), `ls` (style), `label` (légende).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 2, 10)
5 y = x**2
6
7 plt.figure(figsize=(5,5))
8 plt.plot(x, y, c="red", lw=3, ls="--", label="quadratique")
9 plt.plot(x, x**3, label="cubique")
10 plt.title("Courbes quadratique et cubique")
11 plt.xlabel("abscisses")
12 plt.ylabel("ordonnées")
13 plt.legend()
14 plt.show()
```

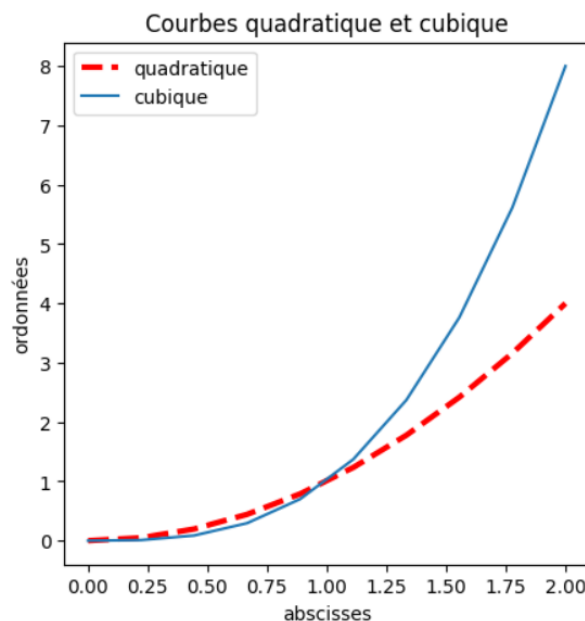


FIGURE 1 – Exemple de tracés avec `plot()`.

11.2 Nuage de points (scatter)

Représente la relation entre deux variables continues. Permet de détecter corrélations, clusters et valeurs aberrantes.

```

1 plt.scatter(x, y, c="blue", marker="o", alpha=0.8, label="points")
2 plt.title("Nuage de points")
3 plt.xlabel("x")
4 plt.ylabel("y")
5 plt.legend()
6 plt.show()

```

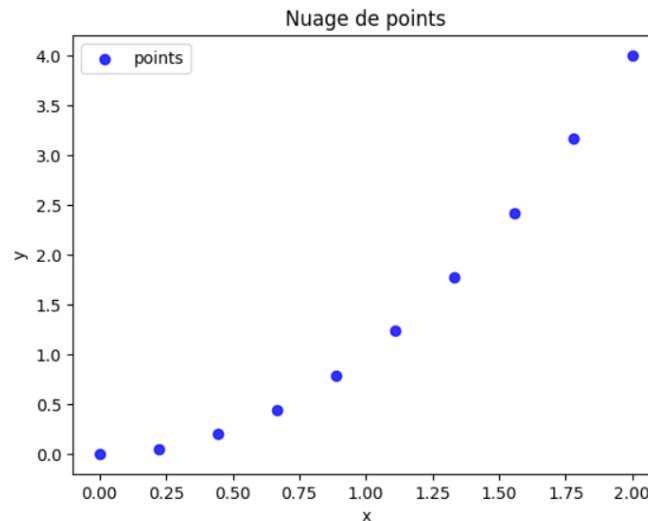


FIGURE 2 – Nuage de points avec `scatter()`.

11.3 Plusieurs graphiques dans une figure

`subplot(rows, cols, index)` découpe la figure en grille pour comparer plusieurs tracés.

```

1 plt.subplot(2, 2, 1)
2 plt.plot(x, y, c="red")
3
4 plt.subplot(2, 2, 2)
5 plt.plot(x, x**2, c="blue")
6
7 plt.subplot(2, 2, 3)
8 plt.plot(x, np.sin(x), c="green")
9
10 plt.subplot(2, 2, 4)
11 plt.plot(x, np.cos(x), c="orange")
12
13 plt.suptitle("Multiples sous-graphiques")
14 plt.tight_layout()
15 plt.show()

```

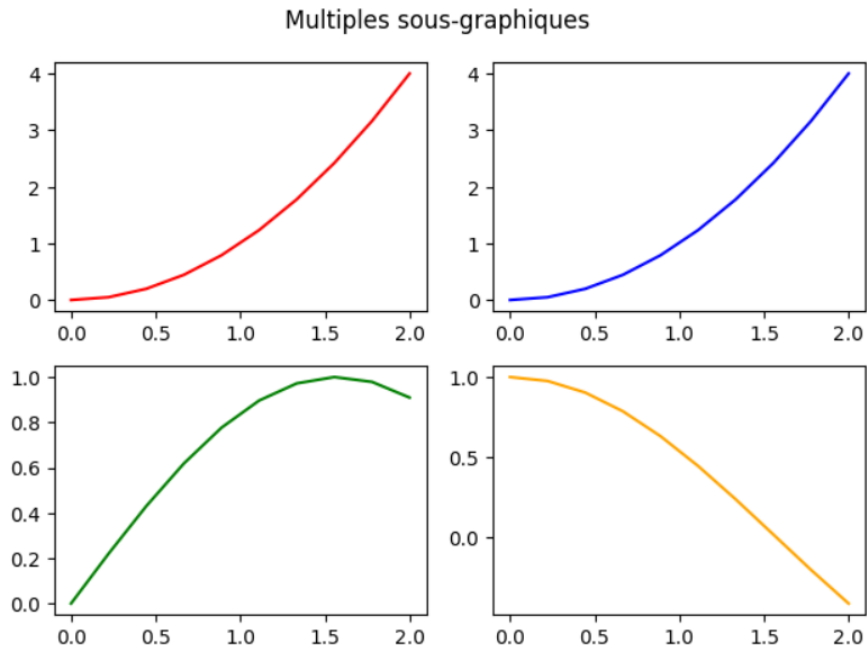


FIGURE 3 – Comparaison de tracés via `subplot()`.

11.4 Interface orientée objet

L'API orientée objet (`fig`, `ax`) donne un contrôle fin (axes, titres, ticks, annotations).

```

1 fig, ax = plt.subplots()
2 ax.plot(x, y)
3 ax.set_title("Approche orientée objet")
4 ax.set_xlabel("x")
5 ax.set_ylabel("y")
6 plt.show()

```

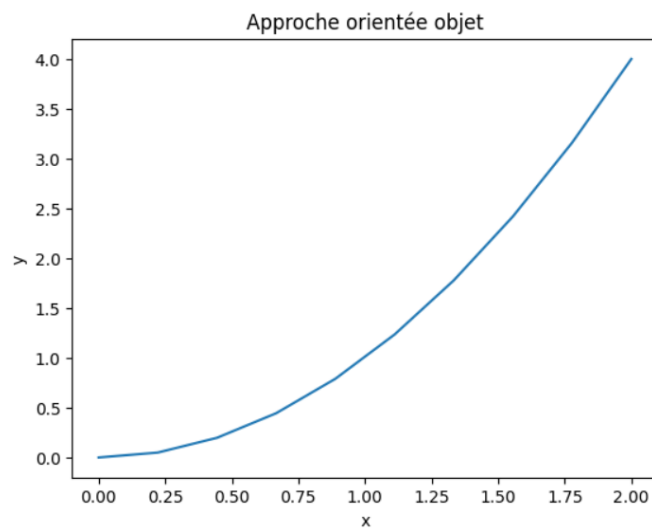


FIGURE 4 – Tracé avec l'API orientée objet.

Sous-figures partageant l'axe x :

```

1 fig, ax = plt.subplots(2, 1, sharex=True, figsize=(6,5))
2 ax[0].plot(x, y, label="x^2"); ax[0].legend()
3 ax[1].plot(x, np.sin(x), label="sin(x)"); ax[1].legend()
4 fig.suptitle("Sous-figures avec abscisses partagées")
5 plt.tight_layout()
6 plt.show()

```

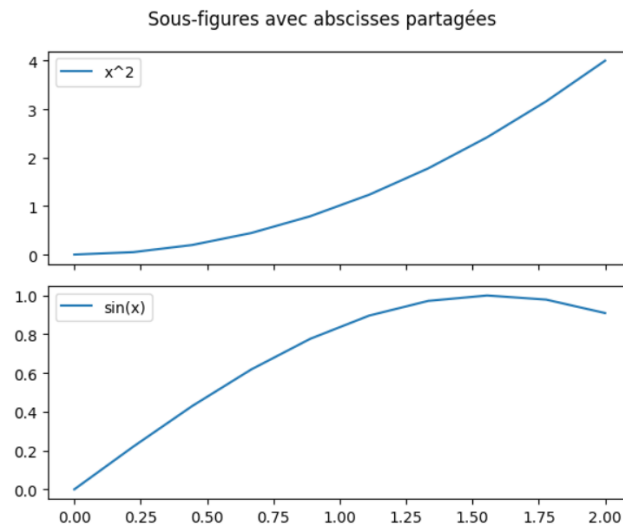


FIGURE 5 – Sous-figures (subplots) avec `sharex=True`.

11.5 Histogrammes

Un histogramme approxime la distribution d'une variable continue en comptant les observations par *bins*.

```

1 data = np.random.randn(1000) # échantillon
2
3 plt.hist(data, bins=30, color="skyblue", edgecolor="black")
4 plt.title("Histogramme d'une loi normale")
5 plt.xlabel("Valeurs")
6 plt.ylabel("Fréquence")
7 plt.grid(alpha=.3)
8 plt.show()

```

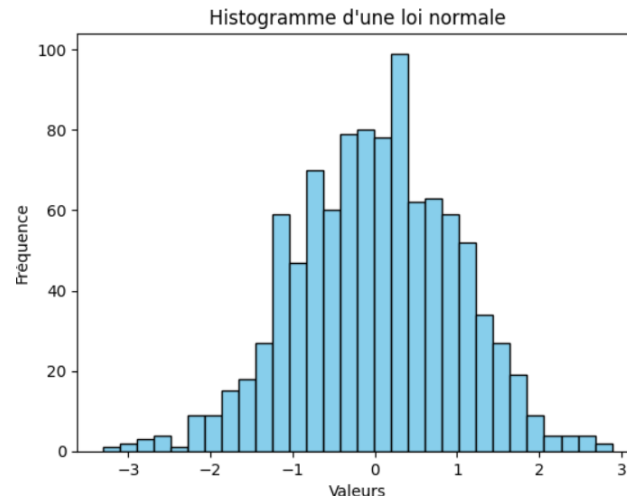



FIGURE 6 – Distribution univariée via `hist()`.

11.6 Histogramme 2D

`hist2d()` : estime la densité conjointe de deux variables (carte de chaleur des fréquences).

```

1 np.random.seed(0)
2 x = np.random.randn(1000)      # 1000 valeurs pour x
3 y = np.random.randn(1000)      # 1000 valeurs pour y
4
5 plt.hist2d(x, y, bins=30, cmap="Blues")
6 plt.colorbar(label="Comptes")
7 plt.title("Histogramme 2D")
8 plt.xlabel("x")
9 plt.ylabel("y")
10 plt.show()

```

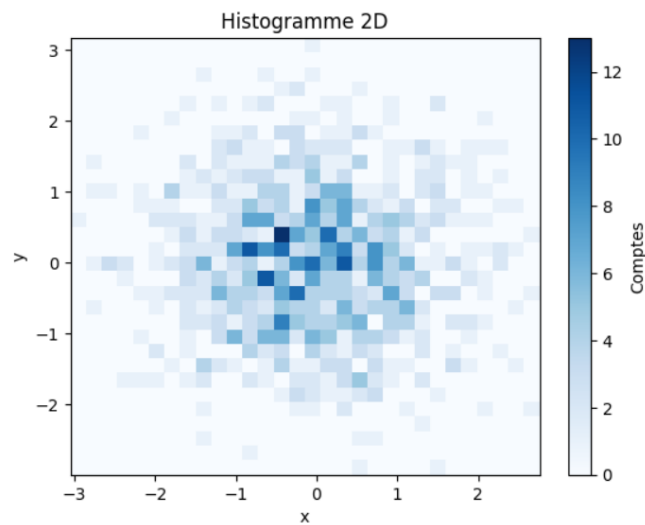


FIGURE 7 – Histogramme 2D représentant la distribution conjointe de deux variables aléatoires.

11.7 Courbes de niveau (contour)

Représente $z = f(x, y)$ par des lignes de même valeur (*isovaleurs*). Très utile pour visualiser des surfaces (p. ex. fonctions de coût).

```
1 X = np.linspace(-5, 5, 80)
2 Y = np.linspace(-5, 5, 80)
3 X, Y = np.meshgrid(X, Y)
4 Z = np.sin(np.sqrt(X**2 + Y**2))
5
6 cs = plt.contour(X, Y, Z, levels=20, cmap="viridis")
7 plt.clabel(cs, inline=True, fontsize=8)
8 plt.colorbar(label="z")
9 plt.xlabel("X"); plt.ylabel("Y")
10 plt.title("Courbes de niveau")
11 plt.show()
```

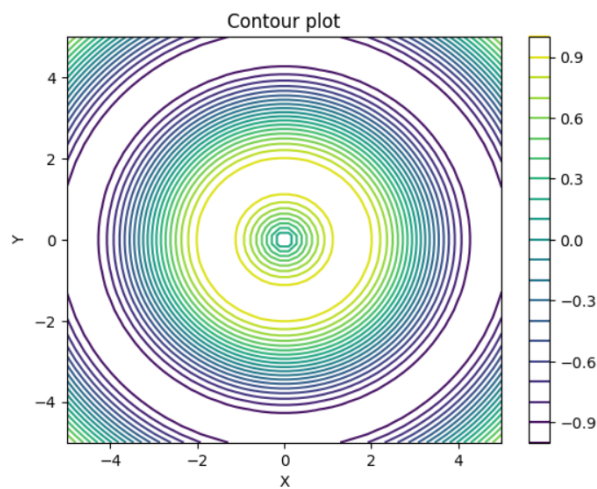


FIGURE 8 – Courbes de niveau pour $z = f(x, y)$.

11.8 Graphiques 3D

Les surfaces 3D aident à explorer des phénomènes dépendant de deux variables (topographie, surfaces de décision).

```
1 from mpl_toolkits.mplot3d import Axes3D
2
3 f = lambda X, Y: np.sin(X) + np.cos(X + Y)
4 X = np.linspace(0, 5, 100)
5 Y = np.linspace(0, 5, 100)
6 X, Y = np.meshgrid(X, Y)
7 Z = f(X, Y)
8
9 fig = plt.figure(figsize=(8,6))
10 ax = fig.add_subplot(111, projection="3d")
11 ax.plot_surface(X, Y, Z, cmap="viridis", linewidth=0, antialiased=True)
12 ax.set_xlabel("X"); ax.set_ylabel("Y"); ax.set_zlabel("Z")
13 ax.set_title("Surface 3D")
```

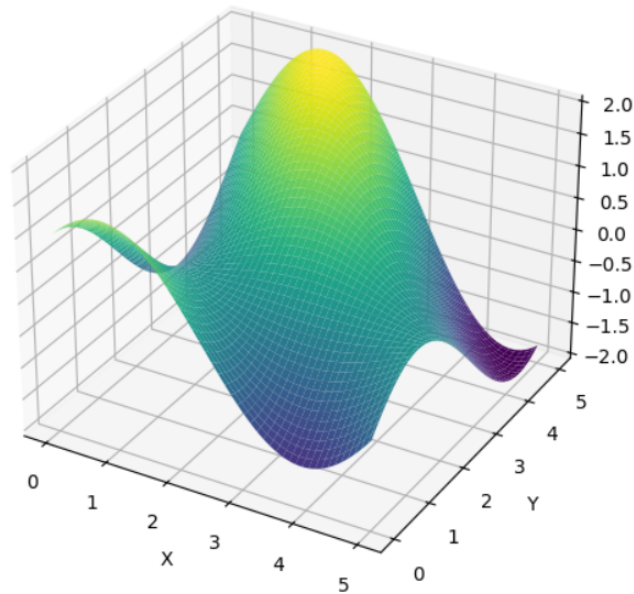


FIGURE 9 – Surface 3D d'une fonction $z = f(x, y)$.

11.9 Matrice de corrélation avec imshow

`imshow()` affiche une matrice comme une image. La matrice de corrélation révèle les relations linéaires entre variables (positives, négatives, nulles).

```

1 data = np.random.randn(200, 5)  # 5 variables
2 corr = np.corrcoef(data.T)
3
4 plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1, interpolation="none")
5 plt.colorbar(label="Corrélation")
6 plt.title("Matrice de corrélation")
7 plt.xticks(range(5), [f"X{i}" for i in range(1,6)])
8 plt.yticks(range(5), [f"X{i}" for i in range(1,6)])
9 plt.show()

```

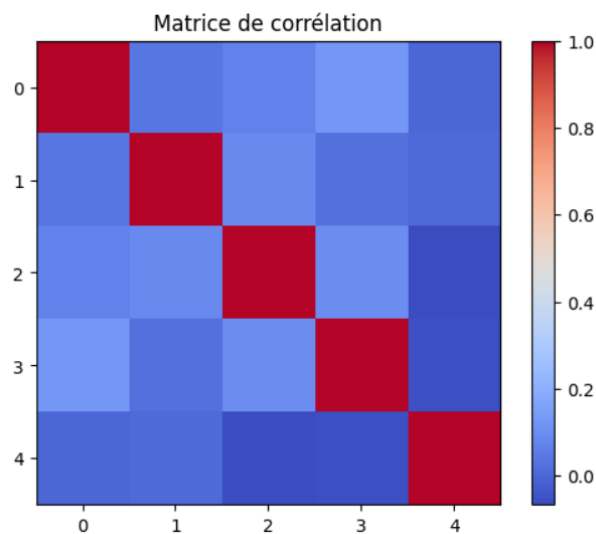


FIGURE 10 – Visualisation d'une matrice de corrélation avec `imshow()`.

12 Visualisation avancée avec Seaborn

Bien que **Matplotlib** soit très puissant, son utilisation peut être limitée et verbeuse. **Seaborn** est une bibliothèque construite sur Matplotlib et Pandas qui offre une syntaxe plus simple, des graphiques plus esthétiques et des fonctions adaptées aux besoins fréquents des data scientists.

- Seaborn simplifie la création de graphiques statistiques ;
- il permet de travailler directement avec les DataFrames de Pandas ;
- les fonctions sont classées par usage (distribution, relation, catégorielle, etc.).

12.1 Exemple avec iris

Nous allons utiliser le dataset `iris`, qui contient des mesures de fleurs de trois espèces différentes.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 # Chargement du dataset iris
7 iris = pd.read_csv("iris.csv")
8
9 # Aperçu des données
10 print(iris.head())
11
12 # Premier scatter plot avec Matplotlib
13 plt.figure()
14 plt.scatter(iris['sepal_length'], iris['sepal_width'])
15 plt.xlabel("Longueur des sépales")
16 plt.ylabel("Largeur des sépales")
17 plt.title("Nuage de points - Iris")
18 plt.savefig("sscatter.png", dpi=150, bbox_inches="tight")
19 plt.show()
```

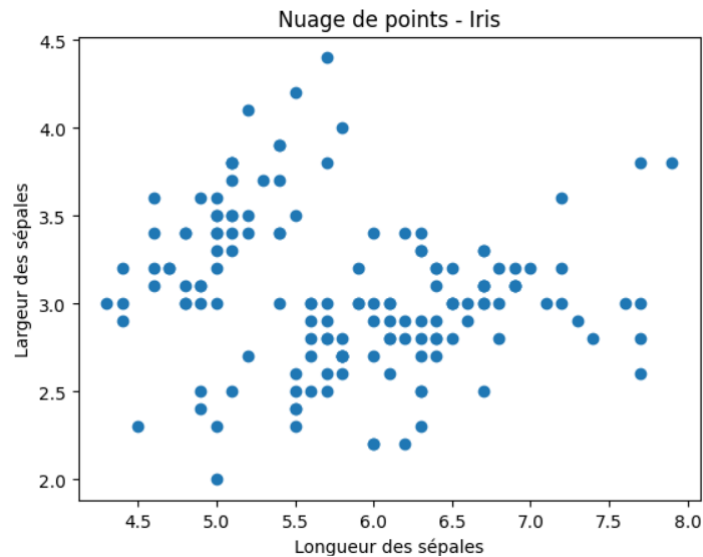


FIGURE 11 – Nuage de points des sépales de l'iris.

12.2 Visualisation avec pairplot

Avec Seaborn, une seule fonction permet de visualiser toutes les relations entre les variables numériques, ainsi que les distributions (histogrammes) sur la diagonale.

```
1 sns.pairplot(iris, hue="species")
2 plt.savefig("pair.png", dpi=150, bbox_inches="tight")
3 plt.show()
```

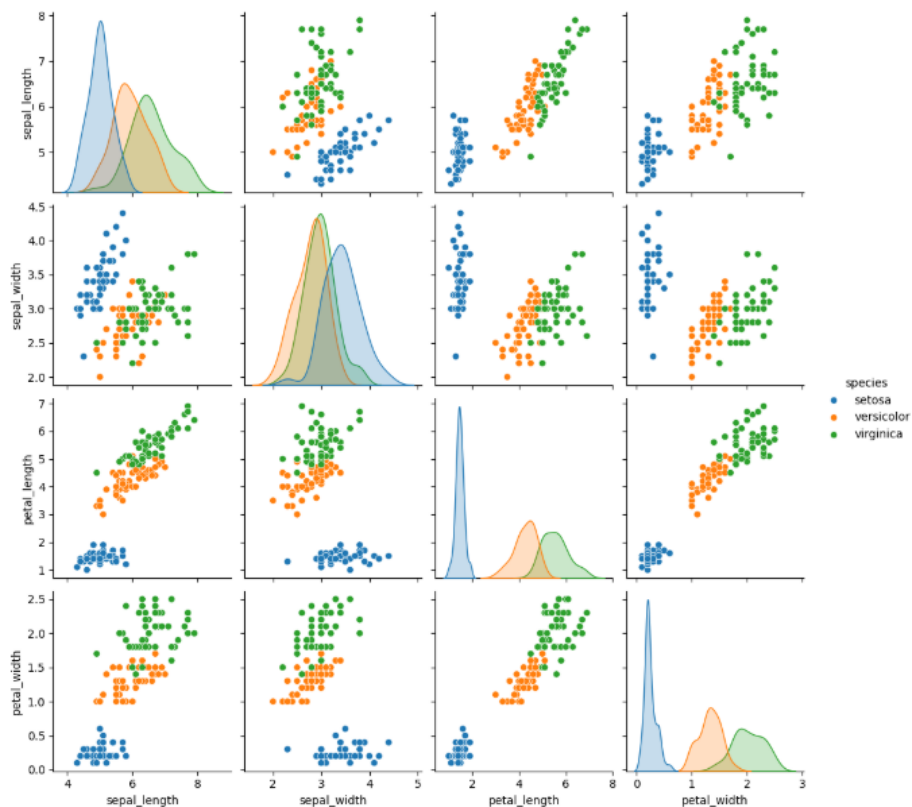


FIGURE 12 – Relations entre les variables de l'iris selon l'espèce (pairplot).

12.3 Exemples avec le dataset Titanic

Seaborn propose aussi des fonctions adaptées aux données catégorielles.

```
1 titanic = sns.load_dataset("titanic")
2
3 # Répartition des âges selon la classe et le sexe
4 sns.catplot(x="pclass", y="age", data=titanic, hue="sex")
5 plt.savefig("sscat.png", dpi=150, bbox_inches="tight")
6 plt.show()
```

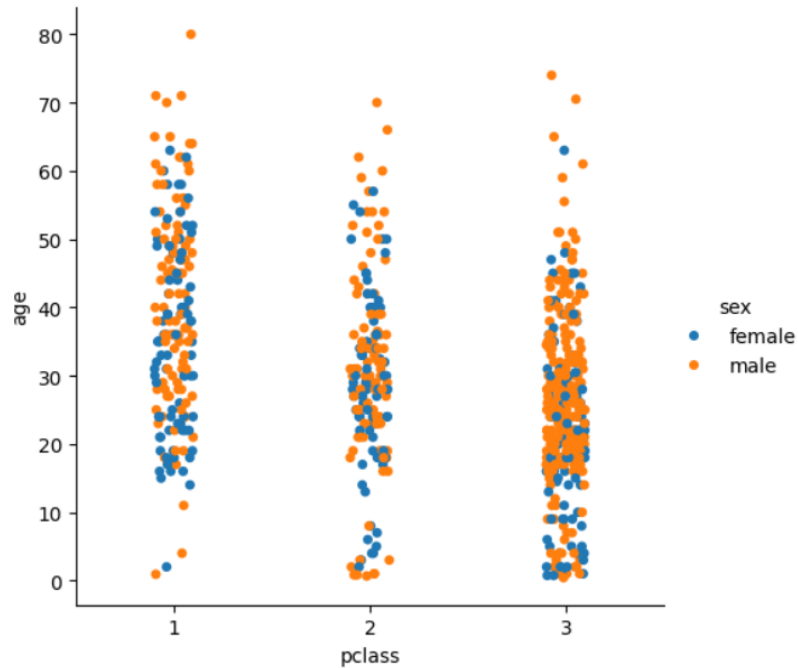


FIGURE 13 – Répartition des âges selon la classe et le sexe (catplot).

12.4 Boxplots

Le boxplot permet de représenter la répartition des données à l'aide de la médiane, des quartiles et des valeurs extrêmes.

```
1 sns.boxplot(x="pclass", y="age", data=titanic, hue="sex")
2 plt.savefig("boxplot.png", dpi=150, bbox_inches="tight")
3 plt.show()
```

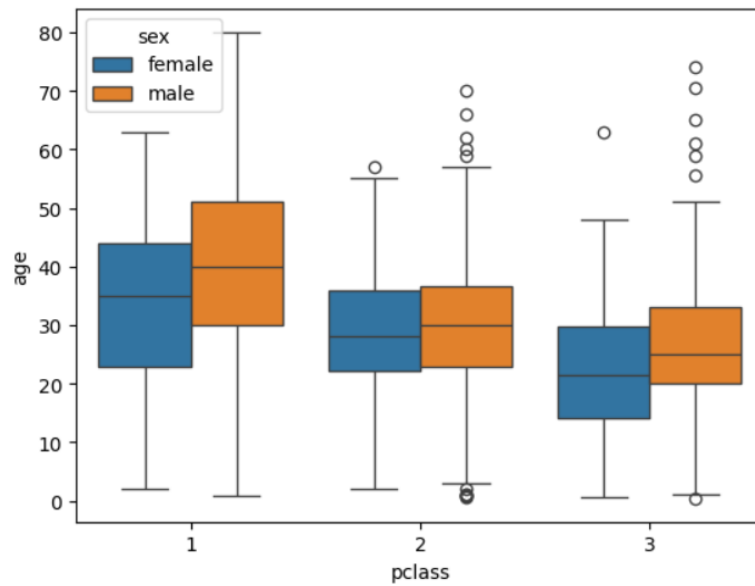


FIGURE 14 – Répartition des âges par classe et par sexe (boxplot).

12.5 Distribution des variables

Pour analyser une variable quantitative, Seaborn propose des fonctions de distribution :

```

1 # Distribution simple
2 sns.histplot(titanic["fare"], kde=True)
3 plt.savefig("hist.png", dpi=150, bbox_inches="tight")
4 plt.show()

```

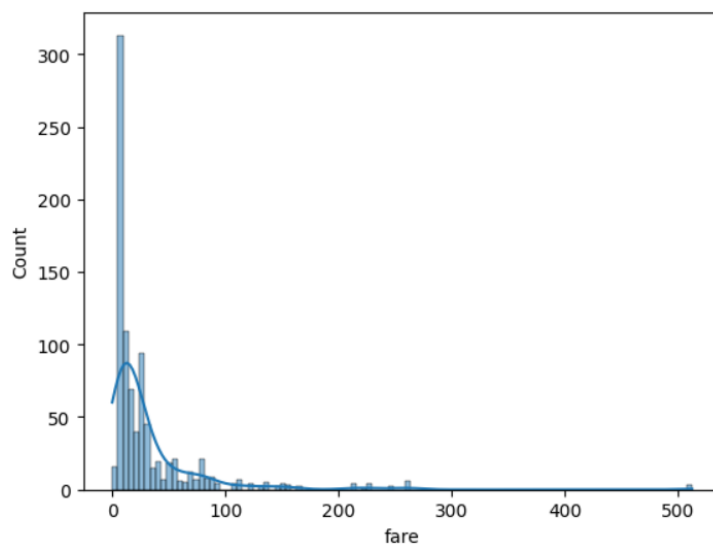


FIGURE 15 – Distribution des tarifs payés par les passagers (histplot avec kde).

12.6 Heatmap des corrélations

La heatmap est très utile pour représenter les corrélations entre variables numériques.

```

1 # Sélection des variables numériques
2 titanic_num = titanic.select_dtypes(include=["number"])
3
4 # Heatmap des corrélations
5 sns.heatmap(titanic_num.corr(), annot=True, cmap="coolwarm")
6 plt.savefig("corr.png", dpi=150, bbox_inches="tight")
7 plt.show()

```

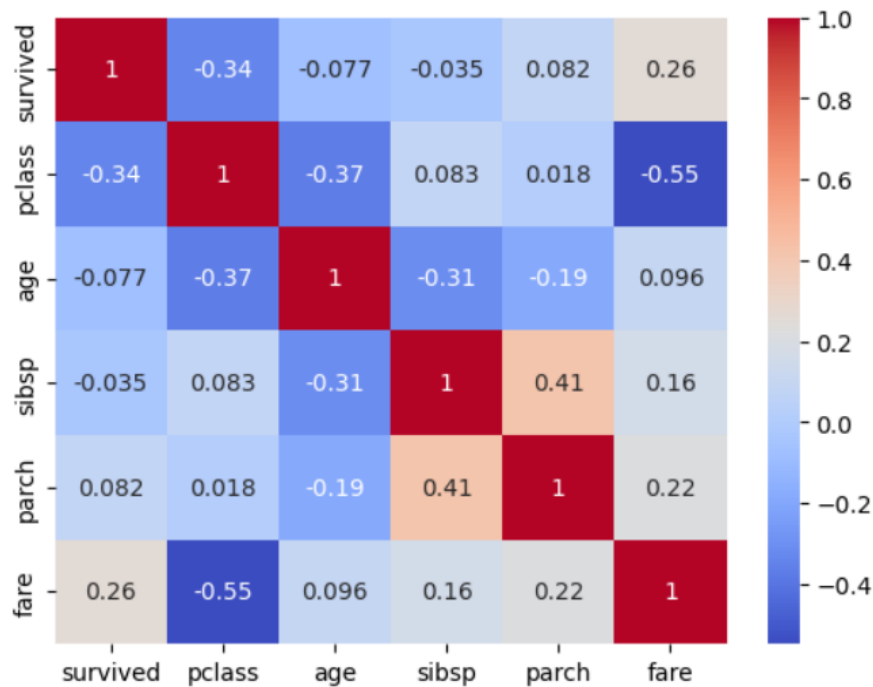


FIGURE 16 – Matrice de corrélation des variables numériques du Titanic.

12.7 Fonctions principales

- `sns.pairplot(data, hue)` : relations entre toutes les variables ;
- `sns.catplot(x, y, data, hue)` : relations catégorielles ;
- `sns.boxplot(x, y, data, hue)` : statistiques descriptives ;
- `sns.histplot(data)` et `sns.jointplot()` : distributions ;
- `sns.heatmap(corr)` : matrice de corrélation.

13 Machine Learning : Apprentissage supervisé

13.1 Introduction

Le **Machine Learning** consiste à donner à une machine la capacité d'apprendre **sans être programmée de façon explicite**. On développe ainsi un modèle à partir de données.

Il existe trois grands types d'apprentissage :

- **Supervisé** : les données sont annotées (on connaît la variable cible).
- **Non supervisé** : les données ne sont pas annotées (ex. clustering).

- **Par renforcement** : la machine apprend en recevant des récompenses/punitions.

13.2 Apprentissage supervisé

- On fournit à la machine des données caractérisées par des variables explicatives et annotées par une **variable cible**.
- Notations :
 - X : features (caractéristiques, variables explicatives).
 - y : label (étiquette, variable cible ou *target variable*).
- Objectif : apprendre à prédire la valeur de y en fonction des features X .

Étapes générales :

1. Fournir des données.
2. Spécifier le type de modèle en choisissant les hyperparamètres (profondeur, nombre de neurones, etc.).
3. Phase d'entraînement : utiliser des algorithmes d'optimisation pour trouver le meilleur modèle.

Problèmes abordés :

- **Régression** : quand y est une variable continue (ex. prédire un prix).
- **Classification** : quand y est une variable discrète ou qualitative (ex. prédire une catégorie).

Résumé : Dataset (données d'entraînement) $\longrightarrow$ Modèle $\longrightarrow$ Prédictions

14 La bibliothèque Scikit-Learn

Scikit-Learn est une bibliothèque Python qui contient la majorité des modèles et algorithmes de Machine Learning. Elle propose une interface unifiée très simple.

- Documentation officielle : <https://scikit-learn.org/stable/>
- Les étapes sont toujours les mêmes :
 1. **Instancier** le modèle (choix de la classe et des hyperparamètres).
 2. **Entraîner** le modèle avec `fit(X, y)`.
 3. **Évaluer** le modèle avec `score(X_test, y_test)`.
 4. **Prédire** avec `predict(X_new)`.

14.1 Syntaxe générale

```
1 # Syntaxe commune à tous les modèles de Scikit-Learn
2
3 model = ModeleChoisi(param1=..., param2=...) # instantiation
4 model.fit(X, y)                               # entraînement
5 score = model.score(X_test, y_test)           # évaluation
6 y_pred = model.predict(X_new)                 # prédiction
```

Tous les modèles de Scikit-Learn suivent la même interface (`fit`, `score`, `predict`).

Régression Linéaire

```
1 from sklearn.linear_model import LinearRegression
2
3 model = LinearRegression()
4 model.fit(X, y)
5 print("Score:", model.score(X, y))
6 y_pred = model.predict(X_test)
```

Régression Logistique

```
1 from sklearn.linear_model import LogisticRegression
2
3 model = LogisticRegression()
4 model.fit(X, y)
5 print("Score:", model.score(X, y))
6 y_pred = model.predict(X_test)
```

Support Vector Classifier (SVC)

```
1 from sklearn.svm import SVC
2
3 model = SVC()
4 model.fit(X, y)
5 print("Score:", model.score(X, y))
6 y_pred = model.predict(X_test)
```

Random Forest Classifier

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier()
4 model.fit(X, y)
5 print("Score:", model.score(X, y))
6 y_pred = model.predict(X_test)
```

15 Évaluation et amélioration des modèles

15.1 Trainset et Testset

En Machine Learning, il est fondamental de ne pas tester un modèle sur les mêmes données que celles utilisées pour l'entraînement. En effet, cela conduirait à un **sur-apprentissage** (overfitting).

- On divise le dataset en deux parties :
 - **Train set** (80%) : utilisé pour entraîner le modèle.
 - **Test set** (20%) : utilisé pour évaluer le modèle sur des données jamais vues.

```
1 from sklearn.model_selection import train_test_split
2
3 # Découpage des données en 80% train / 20% test
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=0
6 )
```

```

7
8 model.fit(X_train, y_train)
9 score = model.score(X_test, y_test)
10 print("Performance sur le test set :", score)

```

15.2 Validation Set et Cross Validation

Parfois, on découpe en trois parties :

- **Train set** : pour l'apprentissage du modèle.
- **Validation set** : pour ajuster les hyperparamètres.
- **Test set** : pour l'évaluation finale.

Une alternative très utilisée est la **validation croisée (cross-validation)**. L'idée est de découper les données en k parties, puis d'entraîner et tester le modèle k fois, en changeant la partie utilisée comme test.

```

1 from sklearn.model_selection import cross_val_score
2
3 from sklearn.ensemble import RandomForestClassifier
4 model = RandomForestClassifier()
5
6 scores = cross_val_score(model, X, y, cv=5) # 5-fold CV
7 print("Scores :", scores)
8 print("Score moyen :", scores.mean())

```

15.3 Validation Curve

La validation curve permet de tester l'influence d'un hyperparamètre particulier sur la performance du modèle. Exemple : le paramètre `max_depth` pour une forêt aléatoire.

```

1 from sklearn.model_selection import validation_curve
2 import numpy as np
3
4 param_range = np.arange(1, 11)
5 train_scores, test_scores = validation_curve(
6     RandomForestClassifier(),
7     X, y,
8     param_name="max_depth",
9     param_range=param_range,
10    cv=5
11 )

```

Cela permet de voir si le modèle est **sous-appris** (underfitting) ou **sur-appris** (overfitting).

15.4 GridSearchCV

GridSearchCV teste automatiquement toutes les combinaisons d'hyperparamètres d'un modèle. Il retourne la meilleure configuration selon la validation croisée.

```

1 from sklearn.model_selection import GridSearchCV
2

```

```

3 param_grid = {
4     "n_estimators": [50, 100, 200],
5     "max_depth": [3, 5, 10]
6 }
7
8 grid = GridSearchCV(
9     RandomForestClassifier(),
10    param_grid,
11    cv=5
12 )
13
14 grid.fit(X, y)
15 print("Meilleurs paramètres :", grid.best_params_)
16 print("Meilleur score :", grid.best_score_)

```

15.5 Learning Curve

La courbe d'apprentissage montre l'évolution des performances du modèle en fonction de la quantité de données disponibles pour l'entraînement. Cela permet de vérifier si :

- Le modèle souffre de sous-apprentissage (performances faibles même avec beaucoup de données).
- Le modèle souffre de sur-apprentissage (forte différence entre train et test).

```

1 from sklearn.model_selection import learning_curve
2 import matplotlib.pyplot as plt
3
4 train_sizes, train_scores, test_scores = learning_curve(
5     RandomForestClassifier(),
6     X, y,
7     cv=5,
8     train_sizes=np.linspace(0.1, 1.0, 10)
9 )
10
11 plt.plot(train_sizes, train_scores.mean(axis=1), label="Train")
12 plt.plot(train_sizes, test_scores.mean(axis=1), label="Test")
13 plt.xlabel("Taille de l'échantillon")
14 plt.ylabel("Score")
15 plt.legend()
16 plt.show()

```

- **Train/Test split** : séparer entraînement et test.
- **Cross Validation** : robustesse en validant sur plusieurs découpages.
- **Validation Curve** : tester l'impact d'un hyperparamètre.
- **GridSearchCV** : trouver la meilleure combinaison d'hyperparamètres.
- **Learning Curve** : observer l'évolution des performances selon la taille des données.